

编译原理实验指导手册

Clang 静态检测项的开发与自动测试

一、阶段一：Clang 静态检测项开发

（一）相关概念

1. 静态检测

静态检测，是指在不运行程序的情况下，对源代码进行分析，发现其中可能存在的缺陷、风险或不符合规范的写法。它通常会检查代码的语法结构、变量使用、类型转换、控制流、数据流、函数调用、宏和头文件等内容，帮助开发人员在编译或运行之前尽早发现问题。

静态检测项，则根据某一具体检测规则设计而成，在扫描代码时，该检测项只会捕获违反该规则的缺陷。在本次实验中，我们使用的规则集是 **MISRA C2012**，使用 LLVM/Clang 技术栈实现检测项的开发。

2. LLVM/Clang

LLVM 是一个庞大的模块化编译器基础设施框架，而 Clang 是 LLVM 项目中的核心子项目，主要作为 C/C++/Objective-C 语言的编译器前端。

Clang 不仅可以作为编译器使用（将.c 或.cpp 文件编译为可执行程序），更是一个高度模块化的库。它将编译过程拆解为词法分析、语法分析、语义分析等多个独立步骤，并开放了丰富的接口。这允许开发者在编译的中间阶段介入，获取程序的抽象语法树（AST）等信息，因此我们可以基于 Clang 编写检测项，在编译过程中自动执行静态检测。

（二）基础环境搭建

1. 准备工具

构建工具 cmake 3.28.0

编译工具 Visual Studio 17 2022

2. 克隆 llvm 源码

```
git clone --branch llvmorg-17.0.5 --depth 1 https://github.com/llvm/llvm-project.git
```

注意放置在全英文路径下

3. cmake 构建

- (1) llvm-project 目录下新建 build 文件夹
- (2) cmd 进入 build 目录，运行

```
cmake -DLLVM_ENABLE_PROJECTS=clang -G "Visual Studio 17 2022" -A x64  
-Thost=x64 ..\llvm
```

若构建成功，最底部的输出是：

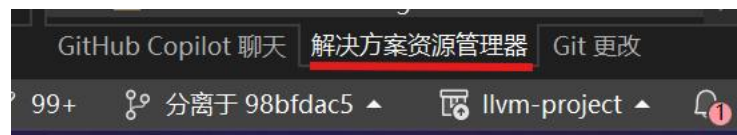
```
-- Configuring done (120.6s)  
-- Generating done (26.9s)  
-- Build files have been written to: D:/Projects/llvm-project/build
```

4. 编译 Clang

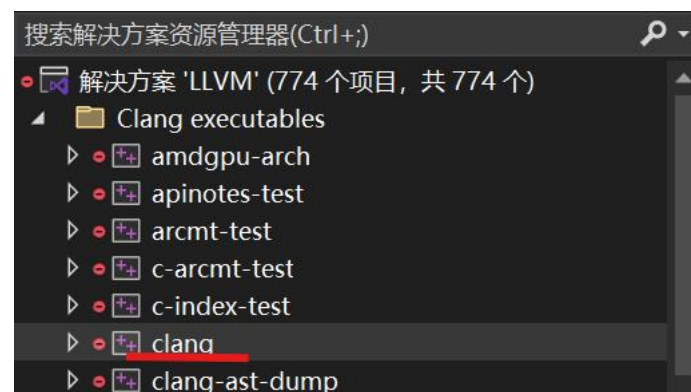
- (1) 在 build 文件夹里找到生成的 LLVM.sln，用 visual studio 打开
- (2) 在顶部选择生成方式为 Release x64



- (3) 点击右下角的“解决方案资源管理器”



- (4) 找到 Clang executables 下的 clang，右键 clang 点击“生成”



编译中内存和 cpu 占用高容易崩溃，建议关闭其他软件。由于是初次编译，耗时较长，耐心等待。期间关注左下角“错误列表”是否报告错误，不用管任何警告。

（5）编译成功显示：

```
===== 生成: xxx 成功, 0 失败, 0 最新, 0 已跳过 =====  
===== 生成 于 xxx 完成, 耗时 xxx 小时 =====
```

注意一定要是 0 失败才证明编译成功。

现在已经拥有了编译好的 `clang.exe`，它的位置在 `build\Release\bin\clang.exe`，记住这个路径。

5. 验证

`clang.exe` 的路径 `--version`

将输出：

```
clang version 17.0.5 (https://github.com/llvm/llvm-project.git 98bfdac5ce82d1679f8af9a57501471812ab68d7)  
Target: x86_64-pc-windows-msvc  
Thread model: posix  
InstalledDir: D:\Projects\llvm-project\build\Release\bin
```

（三）检测项开发

1. 选择开发的规则

（1）MISRA C 简介

MISRA C 是由汽车工业软件可靠性协会（Motor Industry Software Reliability Association）制定的一套 C 语言安全编码规范，主要面向嵌入式与安全关键系统开发，如汽车电子、航空航天、工业控制等领域。

其核心目标是通过限制危险或易产生歧义的 C 语言写法，降低程序中的潜在缺陷，提高代码的安全性、可移植性和可维护性。MISRA C 不仅规定了编码风格，还针对类型转换、指针操作、控制流、预处理器使用等方面提出了严格要求，因此被广泛应用于静态代码分析工具与高可靠软件开发流程中。

目前行业内应用最广泛、生态最成熟的版本仍然是 MISRA C:2012。虽然 2023 年、2025 年又发布了更新版本与修订内容，但由于新版发布时间较短，相关工具链、企业规范及工程实践尚未完全完成迁移，因此 MISRA C:2012 依然是当前工业界与学术研究中最主流、最稳定的参考标准。本项目采用 MISRA C:2012 作为主要依据，既能够保证规范的先进性，也具有较好的工程适用性与行业代表性。

（2）规则选择

本实验为单人独立完成。每人在“MISRA C 分组.xlsx”中选择一组规则（每组规则 2-3 条），并独立完成该组内全部规则的检测项开发。由于规则数量有限，允许不同学生选择相同规则组，同一规则组最多可被 6 人选择；选择相同规则组并不代表组队，代码、报告均需独立提交。

2. 编写检测项

Clang\lib\StaticAnalyzer\Checkers 下，是 Clang 自身支持的检测项。在这个文件夹下，添加某条规则对应的 cpp 文件，规定命名为 MISRAC_1_1_Checker.cpp，具体序号根据规则 id 而定。这个 cpp 文件就负责此条规则的检测。相关开发资料见附件。

在该文件中编写检测逻辑，这其中就要使用 AST、控制流、数据流分析等相关知识。开发思路如下：

（1）分析规则描述

明确该规则限制的是哪一类代码行为。例如，某些规则关注的是**语法层面的结构**，如是否使用了某类语句、某类表达式或某类声明；某些规则关注的是**类型语义**，如表达式两侧类型是否兼容、是否发生了危险的隐式转换；还有一些规则关注的是**数据流或控制流**行为，如变量是否在初始化前被读取、资源是否在所有路径上被正确释放。不同规则对应的分析方式不同，因此在实现前必须先判断它属于哪一类检测问题。

（2）实现检测项

在实现检测项时，应优先判断规则适合使用哪种分析粒度。对于简单、局部、结构明确的规则，可以基于 AST 节点实现，例如检查特定语句、表达式或声明是否出现；对于依赖类型信息的规则，需要结合 clang 的类型系统，读取 QualType、类型限定符、隐式转换节点等信息；对于依赖执行路径、变量状态或资源生命周期的规则，则应使用 Clang Static Analyzer 的路径敏感能力，在不同控制流路径上传播和更新程序状态。

因此，在开发静态检测项时，应始终围绕一个核心问题展开：这条规则在程序中表现为什么代码模式，clang 能从哪些 AST、类型信息或路径状态中识别这

种模式。只有先完成规则到程序结构的映射，再编写 **Checker**，才能避免检测逻辑停留在字符串匹配或表面语法判断层面，从而提高检测项的准确性和可维护性。

3. 注册、构建与编译

(1) 添加到构建目录

打开 `clang/lib/StaticAnalyzer/Checkers/CMakeLists.txt`，在 `add_clang_library(clangStaticAnalyzerCheckers ...)` 的源文件列表中添加 **Checker** 文件名，如 `MISRAC_1_1_Checker.cpp`。这一步是为了将该文件添加到构建目录中，这样检测项才会在编译时被真正地编译进去。

(2) 注册检测项

打开 `clang/include/clang/StaticAnalyzer/Checkers/Checkers.td`，该文件是检测项的注册表，只有在注册表中完成检测项的注册，`clang` 才能识别到它。

注册一个包，代表规则集：

```
def MISRARules : Package<"MISRA">;
```

将你的检测项注册到这个包中：

```
let ParentPackage = MISRARules in {  
  def MISRAC_1_1 : Checker<"R_1_1">, Documentation<NotDocumented>;  
  //...其他检测项  
}
```

可注意到 `Checkers.td` 这个文件中还有许多“长相类似”的包和检测项，这就是前述说到的“`clang` 本身支持的静态检测能力”。

蓝色高亮部分的包名（**规则集名**）和检测项名（**规则 id**），他们将用于检测项的启动命令。

(3) 检测项挂载

完成了注册表的填写后，需要在对应的检测项 `cpp` 文件中，进行“双向绑定”，以便让 `clang` 明白，根据上述包名和检测项名启用某检测项时，该去找哪个文件。

在 `MISRAC_1_1_Checker.cpp` 末尾添加：

```
void ento::registerMISRAC_1_1(CheckerManager &mgr) {  
    mgr.registerChecker<MISRAC_1_1_Checker>();  
}  
  
bool ento::shouldRegisterMISRAC_1_1(const CheckerManager &mgr) {  
    return true;  
}
```

注意高亮部分是该文件内的 Checker 类，若你命名的类名不同，记得在此处更换。

（4）重新构建

由于修改了 CMakeLists.txt，需进入 build 目录重新运行 cmake 命令进行构建，和之前的构建命令一样：

```
cmake -DLLVM_ENABLE_PROJECTS=clang -G "Visual Studio 17 2022" -A x64  
-Thost=x64 ../llvm
```

（5）编译

回到 Visual Studio 的解决方案中，依旧在 Clang executables 下找到 Clang，右键生成。编译成功后，最新的 clang.exe 已经拥有了你自定义的静态检测项。

二、阶段二：自动化检测与缺陷修复

（一）安装 nanobot

解压提供的源码，进入 nanobot-main 目录，在该目录打开命令提示符窗口输入 `pip install -e .`（python 3.9+）

安装完成后若无报错，则进入以下流程：

1、初始化：

在命令行窗口输入 `nanobot onboard`

2、在用户目录下的 `./nanobot/config.json`（例如：`C:\Users\你的 win 用户名`）修

改 json 配置，以下以 deepseekv4 为例：

找到 providers 配置项，修改 apiKey 和 apiBase

```
"deepseek": {  
  "apiKey": "你的 apikey",  
  "apiBase": "https://api.deepseek.com/",  
  "extraHeaders": null,  
  "extraBody": null  
},
```

找到 agents 配置项，修改 model 为如下

```
"agents": {  
  "defaults": {  
    "workspace": "~/nanobot/workspace",  
    "model": "deepseek-v4-pro",  
    "provider": "auto",
```

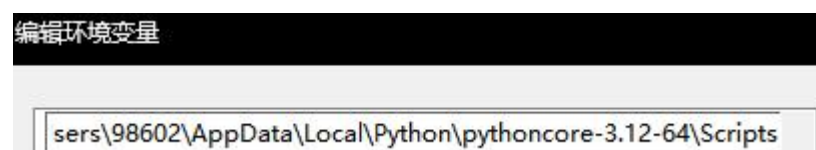
修改好后保存

3、命令行窗口输入 nanobot agent 运行 nanobot

```
C:\Users\98602>nanobot agent  
🐼 Interactive mode (deepseek-v4-pro) - type exit or Ctrl+C to quit  
You: 你好  
🐼 nanobot  
你好！我是 nanobot 🐼，有什么可以帮你的吗？
```

若有回复则证明安装完成

若遇到找不到 nanobot 命令的提示，可通过 where python 找到 python 安装目录，确保 python 文件夹中的 Scripts 在环境变量中，例如图中所示



（二）脚本使用

启动方式可查看工具目录中的 `readme` 文件。

注 1：检测结果中出现少量误报与漏报是正常现象。这是因为 `c` 语言本身具有较强的灵活性，在复杂的宏展开、条件编译、跨函数调用、路径组合及类型推导时难以精确匹配。因此不必追求完全零误报零漏报，能够稳定识别典型违规场景，能够避免明显错误的报告即可。不过如有误报漏报情况，应在实验报告中说明，并注明出现误报漏报的原因。

注 2：可使用下述命令获取 `clang` 的原始缺陷输出

`clang.exe` 路径 `--analyze -Xanalyzer -analyzer-checker=规则集.规则 id -o NUL` 测试用例文件

```
\自动化工具\用例库\GJB8114>"C:\Users\86133\Desktop\staticAnalyzer\bin\clang.exe" --analyze -Xanalyzer -analyzer-checker=GJB8114.R_1_1_5 -o NUL R_1_1_5_violate.cpp
R_1_1_5_violate.cpp:11:2: warning: Value stored to 'a2' is never read [deadcode.DeadStores]
   11 |         a2 = a1 + BLOCKDEF;
      |         ^
R_1_1_5_violate.cpp:13:2: warning: Value stored to 'a4' is never read [deadcode.DeadStores]
   13 |         a4 = a3 + BLOCKDEF;
      |         ^
R_1_1_5_violate.cpp:18:9: warning: 禁止#define被重复定义。 所在函数: Global Scope [GJB8114.R_1_1_5]
   18 | #define BLOCKDEF 2U
      |         ^
3 warnings generated.
```

前两项是 `clang` 原生提供的静态检测功能

三、实验提交要求

将以下内容打包为 `zip` 提交，命名为“学号-姓名-规则组号-编译原理实验报告”，示例“2026123456-张三-1-编译原理实验报告”。

1. 阶段一源文件：规则组内全部检测项 `cpp`
2. 阶段二产物：
 - （1）“用例库”里生成的测试用例源文件
 - （2）“用例库副本”里完成修复的用例文件
 - （3）“关联关系.txt”
3. 实验报告。内容要求如下：
 - （1）实验过程中，每个步骤的文字描述与截图
 - （2）每条检测项的开发思路

- (3) 实验结果：检测项的测试结果、用例的修复结果
- (4) 遇到过什么问题，最终是如何解决的

附件：开发参考资料

1. https://clang-analyzer.llvm.org/checker_dev_manual.html#values

clang 官方的开发手册，想深入了解开发流程的同学可以阅读此文档。

2. <https://clang.llvm.org/doxygen/classes.html>

clang 的 api 文档。网址默认对应的是 clang 的 Main 分支，即开发中的最新版，然而我们使用的是 clang17，虽然 clang17 是一个较新的稳定版，但 clang 的 api 并没有向后兼容保证，某些接口参数在版本迭代中可能发生变化。但核心基类不会发生改变，官方文档依旧具有参考价值，且查找比较方便。如官方文档有误，具体则以本地头文件为准。

3. <https://clang.llvm.org/docs/LibASTMatchersReference.html>

AST Matcher 文档。AST 匹配是比较常见的检测项类型，其中要用到 AST Matcher，可在此文档中查找。如需分析 AST 结构，可使用命令：

```
clang.exe 路径 -fsyntax-only -Xclang -ast-dump test.c
```